

MAMUZA ENGINEERING

Inspire. Solve. Lead

IoT AWARENESS PROGRAM

4-Week Online Cohort

Session 1: Introduction to IoT & Embedded Systems

Student Notes

“Inspire, Learn, Innovate”

info@mamuzaengineering.com | 0745 927 033

Welcome to Session 1

Welcome to the Mamuza Engineering IoT Awareness Program! Over the next four weeks, you will go from knowing nothing about IoT to building and connecting your own smart device. Today we lay the foundation — by the end of this session you will understand what IoT actually is, what powers it under the hood, and you will make an ESP32 board blink its own LED using real code you wrote and understood.

By the end of Session 1, you will be able to:

- ✓ Explain what IoT is and describe its 4-layer architecture in your own words
- ✓ Explain what an embedded system is and identify its 3 core building blocks
- ✓ Explain the difference between a Microcontroller (MCU) and a Microprocessor (MPU)
- ✓ Identify the ESP32 board and explain why it is used for IoT
- ✓ Install ESP32 board support in the Arduino IDE
- ✓ Read and write a basic C program structure
- ✓ Write, upload, and run your first program: blinking the ESP32's built-in LED

HOW TO USE THESE NOTES

Read each section in order — later topics build on earlier ones. Boxes like this one highlight important tips. Grey code blocks contain real code you can copy into the Arduino IDE. Don't worry about memorising everything today; you will practice these ideas hands-on in every session.

Part 1 — What is the Internet of Things (IoT)?

1.1 A Simple Definition

The Internet of Things (IoT) is a network of everyday physical objects — sensors, machines, appliances, vehicles — that are fitted with electronics and internet connectivity so they can collect data, communicate, and sometimes act, all with little or no human involvement.

In short: IoT gives ordinary “dumb” objects the ability to sense the world, think, and talk to us (or to each other) over the internet.

1.2 IoT in Everyday Life

You are probably already surrounded by IoT devices. A few familiar examples:

- A smart water tank sensor that texts you when the tank is running low
- A fitness band that tracks your steps and heart rate and syncs to your phone
- A smart irrigation system that waters crops automatically based on soil moisture
- A security camera that sends an alert to your phone when it detects motion
- A smart electricity meter (like many prepaid tokens meters) that reports usage to the utility company

1.3 Why IoT Matters

IoT matters because it turns raw physical data into decisions and actions — automatically, in real time, at a distance. This unlocks huge value across industries:

- Agriculture — automated irrigation, livestock tracking, soil and weather monitoring
- Healthcare — remote patient monitoring, smart wearables
- Smart homes — automated lighting, security, energy management
- Industry — predictive maintenance, asset tracking, safety monitoring
- Smart cities — traffic management, smart street lighting, waste management

KEY IDEA

IoT is not really about the internet — it's about connecting the physical world to software so decisions can be made from data. The internet is just the messenger.

1.4 The 4-Layer IoT Architecture

Every IoT system, no matter how complex, is built from the same four layers. Understanding these layers now will make everything else in this course click into place.

Layer	What it Does	Examples
1. Perception(Sensing & Acting)	Interacts directly with the physical world — sensing conditions and taking physical action	Sensors (temperature, soil moisture, motion), actuators (relays, motors, pumps), microcontrollers (ESP32, STM32)
2. Network(Connectivity)	Carries the data collected at the edge to the rest of the system	Wi-Fi, Bluetooth (BLE), LoRaWAN, Cellular IoT, MQTT / HTTP protocols
3. Middleware(Processing & Storage)	Receives, processes, filters and stores incoming device data	Cloud platforms (AWS IoT, ThingsBoard, Adafruit IO), databases
4. Application(User Interface)	Where humans see the data and interact with the system	Dashboards, mobile apps, SMS/email alerts

Picture data flowing upward through these layers, like this:

```

[ Sensor reads soil moisture ]      <-- Perception Layer
    |
    v
[ ESP32 sends data over Wi-Fi ]    <-- Network Layer
    |
    v
[ Cloud platform stores & analyses ] <-- Middleware Layer
    |
    v
[ Farmer sees alert on phone app ] <-- Application Layer

```

WHERE THIS COURSE FOCUSES

In this program, we spend most of our hands-on time in the Perception Layer (Weeks 1-2) before moving to Network and Middleware in later sessions. That means: today, we start with hardware — sensors, actuators, and the “brain” that controls them: the embedded system.

Part 2 — Embedded Systems: The Brain Behind IoT

2.1 What is an Embedded System?

An embedded system is a small, specialised computer built into a larger device to perform one dedicated job — reliably, continuously, and usually without anyone noticing it's a computer at all.

This is different from a general-purpose computer (like your laptop or phone), which runs thousands of different apps for many different purposes. An embedded system is purpose-built for one job.

```
[ Physical World ] <---> [ Sensors / Actuators ] <---> [ Embedded
                                                             Brain
                                                             (MCU/MPU) +
                                                             Firmware ]
```

Everyday examples of embedded systems

- A microwave oven's timer and control panel
- A digital wristwatch
- The ABS braking computer in a car
- A washing machine's wash-cycle controller
- The ESP32 board you will use in this course

2.2 The 3 Core Building Blocks

Every embedded system — no matter how simple or advanced — is made of the same three interacting layers:

A. Hardware (The Brain & I/O)

- Processing Unit — a Microcontroller (MCU) or Microprocessor (MPU)
- Memory — Flash (stores the program) and RAM (stores data while running)
- I/O Ports — physical pins that connect to the outside world

B. Physical Interfaces (Inputs & Outputs)

- Sensors (Inputs) — convert real-world data (temperature, light, motion, pressure) into digital signals the MCU can read
- Actuators (Outputs) — convert electrical signals into physical action (motors, relays, heaters, buzzers)

C. Software / Firmware (The Logic)

- Tightly optimised code, usually written in C or C++, that runs directly on the hardware
- Can run Bare-Metal (one simple infinite loop) or use a Real-Time Operating System (RTOS) for precise multi-tasking

THINK OF IT LIKE A PERSON

Sensors are the senses (eyes, skin), the MCU is the brain, and actuators are the muscles. Firmware is the set of instincts and decisions that connect what is sensed to what is done.

2.3 Key Engineering Characteristics

Embedded design prioritises efficiency and reliability over raw power. Good embedded systems are:

- Task-Specific — built to do one job, and do it well
- Resource-Constrained — small memory (often just Kilobytes), limited storage, modest clock speed
- Ultra-Low Power — many run for months or years on a small battery or solar cell
- Real-Time Responsive — must react within strict timing limits, or the system can fail
- Highly Reliable — expected to run for long periods without freezing or needing a restart

2.4 Basic vs. Advanced Embedded Systems

Feature	Standalone / Basic	Complex / Networked (IoT)
Typical Examples	Digital watch, microwave, simple thermometer	Smart car ECU, industrial robot, smart irrigation hub
Processing Brain	8-bit or 16-bit Microcontroller	32-bit MCU or 64-bit MPU / FPGA
Software	Bare-Metal (infinite while(1) loop)	RTOS or embedded Linux
Connectivity	Isolated — no network	IoT-enabled — Wi-Fi, Bluetooth, CAN bus, LoRaWAN

Part 3 — Microcontrollers vs. Microprocessors

Now that we know what an embedded system is made of, let's zoom in on its “brain”. There are two families of processing chips used in embedded and IoT projects: Microcontrollers (MCUs) and Microprocessors (MPUs). Knowing the difference will help you pick the right chip for the right job — and it explains exactly why we use the ESP32 in this course.

3.1 Core Comparison

Feature	Microcontroller (MCU)	Microprocessor (MPU)
Architecture	All-in-one chip: CPU, RAM, Flash, and I/O all on one piece of silicon	Standalone CPU core only — needs external RAM, storage and I/O chips
Power	Ultra-low: milliwatts (mW). Great for battery use	High: needs watts (W), often needs a cooling fan/heatsink
Software	Bare-metal code or an RTOS (e.g. FreeRTOS)	Full OS: Linux, Windows, Android, ROS
Cost & Complexity	Very low cost (\$1–\$5), simple PCB design	Higher cost (\$15–\$100+), complex multi-layer PCB
Common Boards	ESP32, ATmega328P (Arduino), STM32, RP2040	Raspberry Pi, Intel Core boards, ARM Cortex-A boards

3.2 Their Roles: Edge vs. Gateway

A. Microcontrollers — the “Sensory & Nervous System”

- Sit right at the edge, gathering physical data directly from sensors

- Give deterministic, real-time control — reacting within microseconds (critical for motor control, safety shutoffs)
- Sleep deeply to save power, wake up, take a reading, transmit it, and go back to sleep

B. Microprocessors — the “Brain & Gateway”

- Act as IoT gateways, collecting and combining data from many MCU nodes
- Handle heavy compute — computer vision, mapping (SLAM), local databases, web servers
- Run complex, multi-threaded software stacks such as the Robot Operating System (ROS)

3.3 Hybrid Systems — Using Both Together

Modern advanced products rarely rely on just one type of chip. A drone or smart robot, for example, typically combines both:

- The MPU handles the user interface, camera processing, cloud connectivity, and AI
- The MCU handles fast, time-sensitive hardware tasks — reading sensors, reading encoders, driving motors with PWM

WHY WE START WITH AN MCU

In this program we start with an MCU (the ESP32) because it is affordable, low power, and simple enough to master quickly — while still being powerful enough to connect directly to Wi-Fi, which most MCUs cannot do on their own.

Part 4 — Meet the ESP32

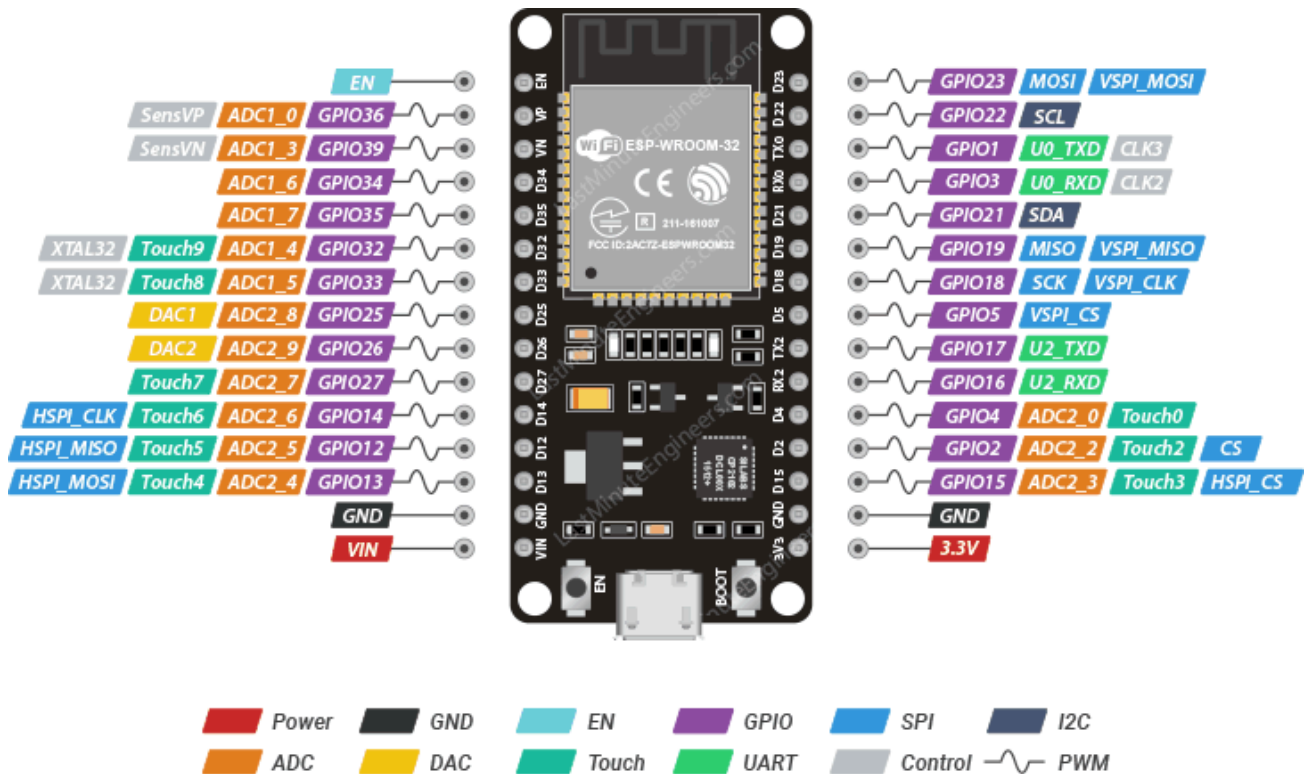
4.1 Why the ESP32?

The ESP32 is the board we will use throughout this program. It is the industry-standard development board for wireless IoT training because it is fast, cheap, and has networking built right in — no extra Wi-Fi module needed.

- 32-Bit Dual-Core Processor — runs up to 240 MHz (over 10x faster than an 8-bit Arduino Uno)
- Integrated Wireless — built-in 2.4 GHz Wi-Fi (802.11 b/g/n) and Bluetooth (Classic + BLE)
- Rich Peripherals — Capacitive Touch, ADC, DAC, UART, SPI, I2C, PWM — all built in
- Power Management — an Ultra-Low-Power (ULP) co-processor for battery-friendly deep sleep

4.2 ESP32 Pinout

The diagram below (to be inserted) shows the ESP32 DevKit pinout — the map of every pin and what it can do. Keep this diagram close by; you will refer to it constantly for the rest of the course.



Critical Hardware Safeguards & Pro-Tips

1. Input-Only Pins — GPIO pins 34, 35, 36 and 39 are input-only. They cannot drive outputs or toggle relays.
2. Pin 12 Boot Behaviour — pulling GPIO 12 HIGH during boot can cause boot loops or flashing failures. Avoid using it for external pull-ups.
3. Current Spikes & Wi-Fi Brownouts — Wi-Fi transmission draws sudden current bursts. Placing a 10 μ F capacitor across VCC and GND stabilises noisy power rails.
4. Dual-Core Use — with FreeRTOS you can assign heavy network tasks (Wi-Fi/HTTP/MQTT) to Core 0 and sensor/motor logic to Core 1.

Part 5 — Setting Up the Arduino IDE for ESP32

Before we can write a single line of code, we need to teach the Arduino IDE how to talk to the ESP32. Follow these steps exactly — do this once and you're set for the rest of the course.

5.1 Install the Arduino IDE

5. Download and install the latest Arduino IDE (2.x) from arduino.cc for your operating system.
6. Open the IDE once to confirm it launches correctly.

5.2 Add the ESP32 Board Package

7. Open the IDE and go to File → Preferences (Windows/Linux) or Arduino IDE → Settings (Mac).
8. In the "Additional Boards Manager URLs" field, paste the ESP32 package URL:

```
https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json
```

9. Click OK to save.
10. Go to Tools → Board → Boards Manager.
11. Search for "esp32" and install the package by Espressif Systems. This may take a few minutes.
12. Once installed, go to Tools → Board and select your specific ESP32 board (e.g. "ESP32 Dev Module").

5.3 Connect and Select Your Port

13. Connect your ESP32 to your computer with a USB data cable (not a charge-only cable).
14. Go to Tools → Port and select the port that appears (e.g. COM5 on Windows, /dev/cu.usbserial-xxxx on Mac).
15. If no port appears, install the CP2102 or CH340 USB driver for your board (check the label on your ESP32 chip).

TROUBLESHOOTING

If upload fails with a timeout, hold the BOOT button on the ESP32 while clicking Upload, and release it once you see 'Connecting...' change to dots. This manually triggers the board's flashing mode.

Part 6 — Programming Foundations: Introduction to C

The ESP32 is programmed using C/C++ through the Arduino IDE. Before diving into embedded-specific concepts, let's cover the basic building blocks of the C language itself.

6.1 What is C?

C is a fast, low-level programming language that gives the programmer close control over how the computer's hardware and memory are used — which is exactly why it is the language of choice for embedded systems.

6.2 Variables and Data Types

A variable is a named container that holds a value. In C, every variable must be declared with a specific data type:

Type	Stores	Example
int	Whole numbers	int count = 5;
float	Decimal numbers	float temperature = 24.5;
char	A single character	char grade = 'A';
bool	True or false	bool isOn = true;

6.3 Basic Syntax Building Blocks

Comments — notes for humans, ignored by the compiler

```
// This is a single-line comment
/* This is a
   multi-line comment */
```

If statements — making decisions

```
if (temperature > 30) {
    Serial.println("It's hot!");
} else {
    Serial.println("Temperature is normal.");
}
```

Loops — repeating actions

```
for (int i = 0; i < 5; i++) {
    Serial.println(i);    // prints 0,1,2,3,4
}
```

Functions — reusable blocks of code

```
int addNumbers(int a, int b) {
    return a + b;
}
```

KEY IDEA

You don't need to master C in one sitting. You just need to recognise these building blocks — variables, if statements, loops, and functions — because every Arduino sketch you write is built from them.

Part 7 — From C to Embedded C

Embedded C extends the C you just learned with a few extra ideas that matter specifically when your code is talking directly to hardware pins and registers.

7.1 Direct Memory-Mapped I/O (the theory behind the magic)

Every physical pin on a chip is connected to a specific memory address called a register. Writing a value to that address turns the physical pin ON or OFF. The Arduino functions you'll use (like `digitalWrite`) do this for you behind the scenes:

```
#include <stdint.h>
// Direct register mapping example for an ARM-based MCU
#define GPIO_PORTA_DATA ((volatile uint32_t*)0x40020014)

int main(void) {
    // Set Bit 0 to HIGH by writing directly to the memory address
    *GPIO_PORTA_DATA = 0x01;
    while (1) {
        // Embedded applications run forever
    }
}
```

}

DON'T WORRY

You will not write raw register code like this in this course — Arduino functions like `pinMode()` and `digitalWrite()` do it for you. It's shown here so you understand what's really happening underneath.

7.2 The 4 Essential Pillars of Embedded C

A. The volatile Keyword

Tells the compiler that a variable's value can change outside the normal program flow — for example, updated by hardware or an interrupt. This stops the compiler from wrongly “optimising away” code that reads that variable.

B. Bit Manipulation (Bitwise Operators)

Registers are just collections of individual bits. These operators let you control one bit at a time:

```
uint8_t register_mask = 0x00;    // Binary: 0000 0000

register_mask |= (1 << 3);        // Turn ON bit 3  -> 0000 1000
register_mask &= ~(1 << 3);      // Turn OFF bit 3 -> 0000 0000
register_mask ^= (1 << 3);        // Toggle bit 3  -> 0000 1000
```

C. Fixed-Width Data Types (stdint.h)

Standard C types like `int` can vary in size across processors. Embedded code needs exact sizes to match hardware registers:

Type	Size	Range
<code>uint8_t</code>	Unsigned 8-bit	0 to 255
<code>int16_t</code>	Signed 16-bit	-32,768 to 32,767
<code>uint32_t</code>	Unsigned 32-bit	0 to 4,294,967,295

D. Interrupt Service Routines (ISRs)

An interrupt pauses the main program the instant a physical event happens (like a button press), quickly runs a small handler function (the ISR), then resumes the main program exactly where it left off.

7.3 Essential Hardware Communication Protocols

Embedded systems talk to sensors and peripherals using three common serial protocols:

Protocol	Wires	Best For
UART	2 (TX/RX), asynchronous	Serial debug output, GPS modules
I2C	2 (SDA/SCL), synchronous	Multiple sensors on one shared bus
SPI	4 (MOSI/MISO/SCLK/CS), synchronous	High-speed displays, SD cards

Part 8 — Anatomy of an Arduino Sketch

A program written for Arduino/ESP32 is called a “sketch”. Every sketch — no matter how complex — is built around exactly two functions:

setup()

Runs exactly once, when the board powers on or resets. Used to initialise things: set pin modes, start serial communication, connect to Wi-Fi, etc.

loop()

Runs over and over, forever, for as long as the board has power. This is where the ongoing work happens: reading sensors, checking conditions, updating outputs.

```
void setup() {
  // Runs ONCE
  pinMode(LED_BUILTIN, OUTPUT);
  Serial.begin(115200);
}

void loop() {
  // Runs FOREVER
  digitalWrite(LED_BUILTIN, HIGH);
  delay(1000);
  digitalWrite(LED_BUILTIN, LOW);
  delay(1000);
}
```

Part 9 — Hands-On: Blink the ESP32's Built-In LED

Time to bring everything together. In this exercise, you will write, upload, and run your very first ESP32 program — making the board's onboard LED blink on and off.

9.1 What You Need

- Your ESP32 development board
- A USB data cable
- Arduino IDE with ESP32 board support installed (Part 5)

9.2 A Note on the Built-In LED Pin

Most ESP32 DevKit boards have a small onboard LED wired to GPIO 2, but this can vary by manufacturer — always check your board's pinout diagram (Part 4.2) to confirm. If LED_BUILTIN isn't recognised by your board package, define the pin number directly, as shown below.

9.3 The Code

```
// Session 1 Project: Blink the ESP32 onboard LED
// Mamuza Engineering - IoT Awareness Program
```

```

#define LED_PIN 2    // Change this if your board's LED is on a different pin

void setup() {
  pinMode(LED_PIN, OUTPUT); // Tell the ESP32 this pin will send OUTPUT signals
  Serial.begin(115200);      // Start serial communication for debugging
  Serial.println("LED Blink program started!");
}

void loop() {
  digitalWrite(LED_PIN, HIGH); // Turn the LED ON
  Serial.println("LED ON");
  delay(1000);                 // Wait 1 second (1000 milliseconds)

  digitalWrite(LED_PIN, LOW);  // Turn the LED OFF
  Serial.println("LED OFF");
  delay(1000);                 // Wait 1 second
}

```

9.4 Line-by-Line: What's Actually Happening

- `#define LED_PIN 2` — creates a name (`LED_PIN`) for pin number 2, making the code easier to read and change later
- `pinMode(LED_PIN, OUTPUT)` — configures GPIO 2 as an output pin, so it can send voltage out
- `Serial.begin(115200)` — opens a communication channel to your computer at 115200 bits per second, so you can see messages in the Serial Monitor
- `digitalWrite(LED_PIN, HIGH)` — sends 3.3V to the pin, lighting the LED
- `delay(1000)` — pauses the program for 1000 milliseconds (1 second) before continuing
- `digitalWrite(LED_PIN, LOW)` — sends 0V to the pin, turning the LED off

9.5 Upload Steps

16. Plug in your ESP32 via USB and select the correct Board and Port (Tools menu).
17. Copy the code above into a new sketch in the Arduino IDE.
18. Click the Upload (→) button.
19. Watch the console at the bottom — wait for “Done uploading.”
20. Watch your ESP32 — the onboard LED should now blink once per second.
21. Open Tools → Serial Monitor (set baud rate to 115200) to see the “LED ON” / “LED OFF” messages print live.

CHECKPOINT

If your LED is blinking and you can see messages in the Serial Monitor, congratulations — you've just written and run your first embedded program! You've officially completed the Perception Layer's most basic building block: controlling an output.

9.6 Try It Yourself

Before Session 2, experiment with the code above:

- Change the delay values — make the LED blink faster or slower

- Make the LED blink in a pattern (e.g. 3 quick blinks, then a long pause)
- Try connecting an external LED with a resistor to a different GPIO pin and control that instead

Session 1 Recap

Today you covered a lot of ground. Here's what you now know:

- ✓ IoT connects physical devices to the internet through 4 layers: Perception, Network, Middleware, Application
- ✓ Embedded systems are dedicated, purpose-built computers made of Hardware, Interfaces, and Firmware
- ✓ MCUs (like the ESP32) live at the edge and are low-power and low-cost; MPUs (like a Raspberry Pi) act as gateways and handle heavy compute
- ✓ The ESP32 is our chosen board for this course because of its built-in Wi-Fi, Bluetooth, and low cost
- ✓ C programs are built from variables, if statements, loops, and functions
- ✓ Embedded C adds volatile, bit manipulation, fixed-width types, and interrupts on top of standard C
- ✓ You wrote, uploaded, and ran your first ESP32 program: the Blink sketch

Looking Ahead to Session 2

In our next session, we move from Local Control to Telemetry: you will connect your ESP32 to Wi-Fi, make your first HTTP request, and start reading real sensor data.